

Communication 1 — Direct Serial Communication

Tutorial Group: Communication

Goal

Build a direct UART serial connection between an Arduino Mega 2560 and an ESP32.

In this tutorial, you will learn how two microcontrollers can exchange serial data even though they use different logic voltages. You will examine the electrical requirements of each direction of communication, build a simple resistor-divider level shifter, and verify communication in both directions.

Learning objectives:

- Connect two microcontrollers using UART serial communication.
- Explain the difference between 5v and 3.3v logic levels.
- Explain why the ESP32 TX signal can connect directly to the Arduino Mega 2560 RX input.
- Explain why the Arduino Mega 2560 TX signal must be reduced before reaching the ESP32 RX input.
- Build a simple 1 k Ω / 2 k Ω resistor-divider level shifter.
- Use a common ground while keeping the two processor power supplies separate.
- Test serial communication in both directions.

What You Need

- Arduino Mega 2560
- ESP32 development board
- Computer(s) with the Arduino and ESP32 IDE
- Two breadboards – one dedicated to the Arduino Mega 2560 and one dedicated to the ESP32, and jumper wires
- 1 $\times$ 1 k Ω resistor
- 1 $\times$ 2 k Ω resistor
- USB cables

If possible, using two laptops will make the tutorial easier, since the Arduino Mega 2560 and ESP32 can be programmed and monitored independently.

⚠ IMPORTANT: The two breadboards are intentionally separate. **Do not connect the VCC/positive power rail of the Mega 2560's 5v breadboard to the VCC/positive power rail of the ESP32's 3.3v breadboard.**

Part 1 — Understand the Interface

The Arduino Mega 2560 and ESP32 can communicate using ordinary UART serial communication, but they do not operate at the same logic voltage. This is an important electrical difference that must be understood before connecting the two processors. The same 5v and 3.3v logic-level principles also apply to other digital signal connections.

For this tutorial, the Arduino Mega 2560 operates at 5v and the ESP32 operates at approximately 3.3v. Both devices use digital signals to represent binary data. A digital LOW is represented by a voltage near 0v, while a digital HIGH is represented by a higher voltage. The important point is that the acceptable voltage for a HIGH depends on the receiving device.

The important point is that 5v and 3.3v are not interchangeable logic levels. A 3.3v output can be acceptable to some 5v inputs, while a 5v output may not be safe for a 3.3v input. Always determine the electrical requirements of both sides before connecting a signal between systems that use different logic voltages. In this tutorial, the ESP32 GPIO

pins are designed for approximately 3.3v operation, so a voltage above the ESP32's specified input limit can damage its GPIO.

The Arduino Mega 2560, on the other hand, is designed to operate with 5v logic. Therefore, simply connecting every corresponding signal between the two processors would not be safe.

The two directions of communication are not electrically identical because the voltage levels produced by the two processors are different. That is why this tutorial uses a direct connection in one direction and a resistor divider in the other. The same level-shifting concept can be used for other digital signals, not just UART.

The signal connections for this tutorial are:

- ESP32 TX2 → Mega RX1
- Mega TX1 → ESP32 RX2 through the resistor divider
- Do not connect TX to TX or RX to RX.

TX means transmit data, and RX means receive data. In a UART connection, the transmitter's TX signal connects to the receiver's RX input.

Because the ESP32's approximately 3.3v HIGH is above the Mega 2560's minimum guaranteed HIGH input threshold, the ESP32 TX signal can connect directly to the Mega RX input.

Data traveling from the ESP32 to the Arduino

The ESP32 produces a HIGH of approximately 3.3v. The Arduino Mega 2560 is operating from a 5v supply, but that does not mean its input must receive 5v to recognize a HIGH. A digital input has a specified HIGH-voltage threshold. If the incoming signal is above that threshold, the Arduino interprets it as a logic HIGH.

For the Mega 2560, the specified minimum HIGH input voltage is approximately 3.0v when operating at 5v. The ESP32's output HIGH is approximately 3.3v, which is above that threshold.

Therefore, the ESP32's 3.3v HIGH is high enough for the Arduino Mega 2560 to recognize as a HIGH.

The LOW level is also compatible because both devices use a voltage near 0v to represent LOW.

Data traveling from the Arduino to the ESP32

The Arduino Mega 2560 can produce a HIGH close to 5v. The ESP32 GPIO is a 3.3v device and must not be connected directly to that 5v signal.

Instead, we need a simple circuit that reduces the Mega 2560's approximately 5v signal to a voltage suitable for the ESP32. For this tutorial, we will use a two-resistor voltage divider.

The divider uses:

- A 1 kΩ between Mega 2560 TX and the ESP32 RX node.
- A 2 kΩ between the ESP32 RX node and GND.

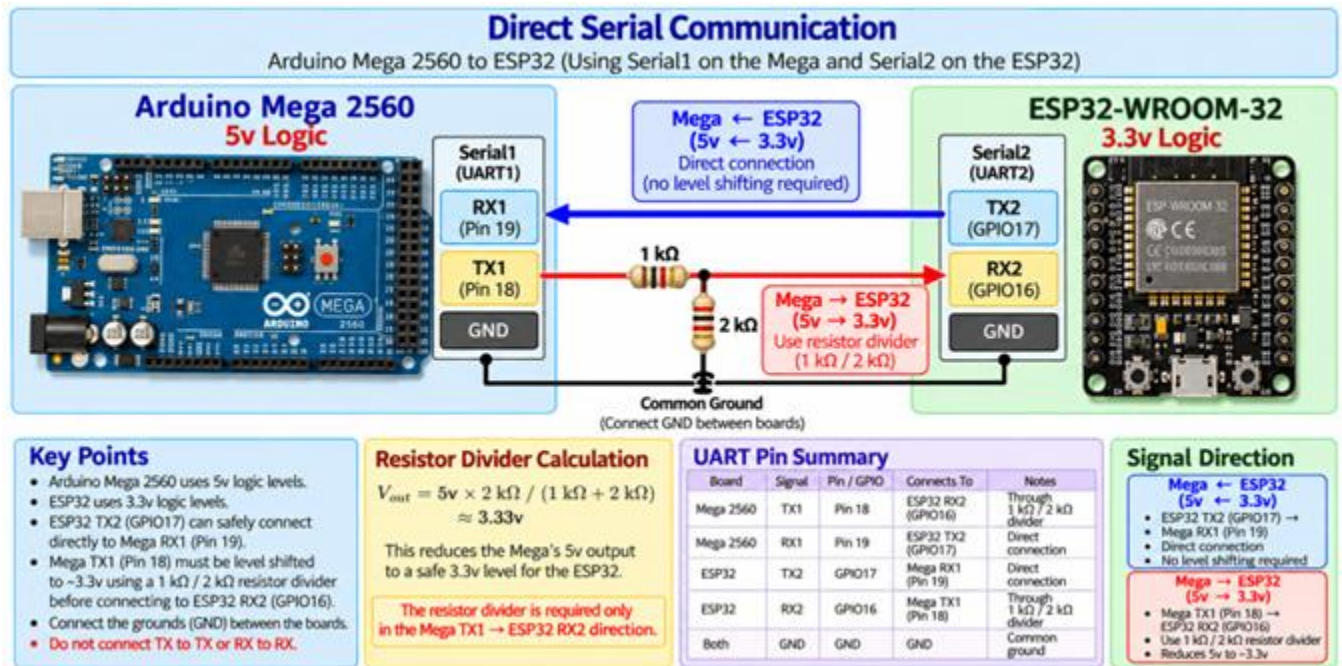
For these resistor values, a 5v input is reduced to approximately 3.33v:

$$5\text{v} \times 2\text{ k}\Omega / (1\text{ k}\Omega + 2\text{ k}\Omega) \approx 3.33\text{v}$$

Important: The resistor divider is required only in the Mega TX1 → ESP32 RX2 direction. Do not place the divider in the ESP32 TX2 → Mega RX1 path.

The important idea is that the divider protects the ESP32 input from the higher-voltage Arduino signal.

Complete interface



DO NOT connect the ESP32 VCC/3.3v supply to the Mega 2560 5v supply

For this tutorial, each microcontroller is powered independently through its own USB connection. The only power-related connection between the two systems is their common GND. It is important that the two processors share a common ground. Connect GND of the ESP32 to the GND of the Mega 2560, but do NOT connect 5v to 3.3v.

The USB connection to the computer is used to power, program, and monitor each development board. It is separate from the UART connection being tested between the Mega and ESP32.

A 3.3v output is not automatically incompatible with a 5v processor. Likewise, a 5v output is not automatically safe for a 3.3v processor. Always determine the electrical requirements of both sides before connecting the two different microcontrollers.

Part 2 — Serial Communications from the Mega to the ESP32

With the electrical interface understood and wired as shown in Part 1, we can perform the first communication test.

For this test, the Arduino Mega 2560 will transmit, the ESP32 will receive, and the UART communication speed will be 9600 baud. The ESP32 will display the received characters through its USB serial connection.

Arduino Mega 2560 — Transmitter

Upload this program to the Arduino Mega 2560.

```
// Arduino Mega 2560 transmitter

void setup()
{
  Serial1.begin(9600);
}

void loop()
{
  Serial1.println("Hello from Mega 2560");
  delay(1000);
}
```

The Mega 2560 sends the message "Hello from Mega 2560" once every second using Serial1.

ESP32 — Receiver

Upload this program to the ESP32.

```
// ESP32 receiver

void setup()
{
    Serial.begin(115200);
    Serial2.begin(9600, SERIAL_8N1, 16, 17);
}

void loop()
{
    while (Serial2.available())
    {
        Serial.write(Serial2.read());
    }
}
```

The ESP32 receives the data through Serial2. It then sends the received characters through its USB serial connection using Serial, allowing you to see the message in the Serial Monitor.

For the ESP32 configuration shown in this tutorial, Serial2 uses GPIO16 for RX2 and GPIO17 for TX2. The wiring diagram and code use these same pins.

Expected Result

Open the ESP32 Serial Monitor at 115200 baud. You should see:

```
Hello from Mega 2560
Hello from Mega 2560
Hello from Mega 2560
```

The communication between the two microcontrollers is running at 9600 baud. The ESP32's USB serial connection to the computer is running separately at 115200 baud so that the received information can be displayed.

Part 3 — Serial Communications from the ESP32 to the Mega

The first test demonstrated communication from the Arduino Mega 2560 to the ESP32. Now test the communication in the opposite direction.

Test ESP32 TX2 → Mega RX1

Modify the programs so that the ESP32 transmits through Serial2 and the Mega 2560 receives through Serial1.

This direction does not require the resistor divider. The ESP32's approximately 3.3v HIGH is sufficient for the Mega 2560's input. Have the ESP32 send a short message such as "Hello from ESP32" through Serial2. Have the Mega receive the message through Serial1 and display it through its USB serial connection.

Verify that the message is received correctly.

Part 4 — Now Let's Send Messages In Both Directions

You have now tested each direction of communication separately. Now modify the programs so that both processors periodically transmit a message identifying themselves. The Mega should transmit "Hello from Mega 2560" and the ESP32 should transmit "Hello from ESP32".

Observe the messages received by each processor and verify that communication works in both directions at the same time.

What You Should Understand

At the completion of this tutorial, you should understand:

- UART is an asynchronous serial communication method.
- UART does not require a shared clock signal.
- The transmitter's TX connects to the receiver's RX.
- Both processors must agree on the baud rate and serial format.
- The Arduino Mega 2560 uses 5v logic.
- The ESP32 uses approximately 3.3v logic.
- A 3.3v ESP32 output can be recognized as HIGH by the Mega 2560's input.
- A 5v Mega 2560 output must not be connected directly to an ESP32 GPIO.
- A simple resistor divider can reduce the Mega 2560's 5v TX signal to approximately 3.3v.
- The 1 k Ω / 2 k Ω divider is used only in the Mega 2560 TX $\rightarrow$ ESP32 RX direction.
- The two processors must share a common ground.
- The two processors must not have their 5v and 3.3v power supplies connected together.
- Each processor should be powered independently.
- Communication speed and logic voltage are separate concepts.

Before Moving On

- ☐ I understand why both processors must use the same baud rate.
- ☐ I can explain the difference between 5v and 3.3v logic.
- ☐ I can explain why the ESP32 TX can connect directly to the Mega 2560 RX.
- ☐ I can explain why the Mega 2560 TX cannot connect directly to the ESP32 RX.
- ☐ I understand that the voltage divider reduces the Mega 2560's approximately 5v signal to approximately 3.3v.
- ☐ I understand why the two processor grounds must be connected.
- ☐ I understand why the 5v and 3.3v power supplies must not be connected together.
- ☐ I can communicate between the Arduino Mega 2560 and ESP32 in both directions.
- ☐ I can explain why both ends of a UART connection must agree on the baud rate.

If you can complete these tasks and explain what you are seeing in the serial connection, you are ready for the next tutorial.